ShopFlow Curriculum · Part 0 — Foundations

Part 0 — Foundations

Professional Edition — Full-Stack Developer Study Roadmap. Everything to set up before Phase 1 begins: environment setup, Linux and terminal fundamentals, Java syntax, Git and GitHub, UML diagramming, and the professional habits — documentation, debugging method, and AI-assisted learning — that carry through the entire roadmap.

6 topics (0–5) · 3–4 weeks · Beginner · 8–12 hrs/wk · 3–4 projects per topic · 3+ documentation sources per topic

How to Use This Roadmap

Every topic below follows the same shape: objectives, required knowledge, core concepts, hands-on exercises, mini projects, common mistakes, official documentation, a self-check quiz, a reflection prompt, and a completion checklist. The four habits below apply across all of them and are not repeated in full inside each topic.

Study Rhythm

- Read the concept content, then do the Hands-On Exercises the same day — they're short by design
- Save Mini Projects for once the concepts feel solid, not as your first exposure to them
- Check off the topic's Completion Checklist honestly before moving on — skipped checkboxes compound

Documentation-First Habit

- Before searching, check the official docs linked at the end of the topic — they're listed for a reason
- Treat Stack Overflow, blog posts, and video tutorials as secondary sources, not the first stop
- Write a short README for every project explaining what it does, why it exists, and how to run it

AI-Assisted Learning Guidelines

- Write the code yourself first, even a rough attempt — don't ask an AI assistant to produce it from a blank file
- Use AI to explain a concept or an error message, not to hand you the final answer
- Once your code works, ask an AI assistant to review it, and require yourself to understand every suggested change before accepting it
- Never commit or merge code you can't explain line by line, regardless of who or what wrote it
- This gets revisited in more depth in Topic 5, once you've built enough real code to practice it on

Getting Unstuck as a Self-Taught Learner

1. Read the exact error message, top to bottom, before searching anything
2. Reproduce the smallest failing case in isolation — strip away everything not related to the bug
3. Check the official docs for the exact method or class name involved
4. Search the exact error text in quotes
5. Ask an AI assistant to explain — not solve — the problem, once you've tried the first four steps
6. If still stuck after 30–45 minutes, write down what you tried, switch to a different subtopic, and return later

> This last habit — time-box and rotate — prevents burnout spirals and is worth writing down as a rule, not just a suggestion. Each topic's Reflection prompt is where you log moments like this.

Topic 0 — Environment Setup

Estimated Duration: 3–4 days

Subtopics: JDK install & version management · verifying the install · JAVA_HOME & PATH · choosing one shell · IntelliJ IDEA setup · first debugger encounter

Nothing else in this roadmap is possible until a JDK, a shell, and an IDE all work together. This topic exists so every later topic can assume that.

Learning Objectives

By the end of this topic, you will be able to: install and verify a working JDK; explain what JAVA_HOME and PATH do and why build tools depend on them; choose one primary shell environment and commit to it for the rest of the roadmap; configure IntelliJ IDEA and run and debug a one-file program.

Required Knowledge

None — this is the starting point of the roadmap.

Core Concepts

Install the JDK

- Install a current LTS JDK (Java 21 or later) — Eclipse Temurin or Oracle's build both work
- macOS/Linux: use SDKMAN (sdkman.io) to install and switch JDK versions — this pays off the moment two projects need different versions

- Windows: install WSL2 and use SDKMAN inside it (recommended), or the official Windows installer as a fallback

Verify the Install

- `java -version` and `javac -version` should print matching version numbers
- If either command isn't found, the JDK's bin folder isn't on PATH yet — this is a PATH problem, not a broken install
- Confirm JAVA_HOME is set and points at the JDK install directory — Maven and Gradle read this variable directly

Choose One Shell Environment

- macOS/Linux: your default terminal (zsh/bash) is already a real Unix shell — nothing else to install
- Windows: install WSL2 and do the entire roadmap inside it — this is the standard setup for Windows developers doing backend/Java work, and Docker later on expects a Linux-like environment anyway
- Git Bash remains a lighter fallback only if WSL2 truly isn't an option — treat it as a fallback, not a parallel track to master

Install and Configure IntelliJ IDEA

- IntelliJ IDEA Community Edition is free and sufficient for this entire roadmap
- Open a single .java file, confirm the Run button works, then confirm the Debug button also works on the same file — you'll use Debug starting in Topic 2
- Enable autosave and turn on line numbers in Settings — small workflow habits that pay off immediately

Hands-On Exercises

1. Run `java -version` and `javac -version` side by side and confirm they match
2. Print JAVA_HOME and confirm it points at your JDK install folder
3. Create a one-line HelloWorld.java in IntelliJ and run it with Run, then again with Debug, stepping over the single line

Mini Projects

First Boot Checklist — Steps: install the JDK → run `java -version` and `javac -version` and confirm they match → confirm JAVA_HOME is set → install IntelliJ IDEA → open a blank project and confirm the Run button works on a one-line program.

Two-JDK Switch Drill — Proves PATH/JAVA_HOME are just pointers you can redirect; this becomes routine once ShopFlow depends on a specific version. Steps: install two JDK versions with SDKMAN → run `sdk list java` to see both → switch with `sdk use java <version>` → confirm java -version changes each time you switch.

Common Mistakes

- Installing the JDK but never adding it to PATH, then assuming the install itself failed
- Mixing WSL2 and native Windows CMD commands in the same session
- Skipping JAVA_HOME because things "seem to work anyway" — this breaks silently later when Maven or Gradle can't find it

Official Documentation

- Oracle: JDK Installation Guide — docs.oracle.com/en/java/javase/21/install/
- SDKMAN — sdkman.io
- Microsoft: WSL Installation Guide — learn.microsoft.com/en-us/windows/wsl/install

Self-Check Quiz

1. What does JAVA_HOME point to, and which tools read it?
2. Which two commands verify a JDK install, and what should be true of their output?
3. Why is WSL2 recommended over plain Windows CMD for this roadmap?
4. What's the difference between Run and Debug mode in IntelliJ?
5. Name one reason SDKMAN stays useful after the first install.

> **Reflection:** In 2–3 sentences, note anything that didn't install cleanly and how you resolved it. This is your first entry in the running "getting unstuck" habit described in the front matter — you'll keep making entries like this through every topic.

Completion Checklist

- [ ] java -version and javac -version print matching versions
- [ ] JAVA_HOME is set and confirmed
- [ ] One shell environment chosen and used going forward
- [ ] IntelliJ IDEA installed; a one-file program runs and debugs successfully
- [ ] Quiz self-checked and reflection written

> **Bridge to Part A:** Every Spring Boot project in Part A runs on this same JDK, through this same terminal, inside this same IDE — nothing set up here gets thrown away.

Topic 1 — Linux & Terminal Fundamentals

Estimated Duration: 4–5 days

Subtopics: shell navigation · Linux filesystem basics · file operations · permissions · running programs · PATH & environment variables · piping & redirection · command-line debugging method

Every tool from here forward — javac, git, npm, docker, mvn — is run from this terminal. If typing commands still feels unfamiliar, slow down here; everything downstream assumes it's second nature.

Learning Objectives

Navigate and manipulate the filesystem from the command line without a GUI; explain the basic Linux filesystem layout and the difference between absolute and relative paths; read and change basic file permissions; compile and run a Java program entirely from the terminal; use pipes and redirection to filter and save command output; apply a repeatable method for isolating a command-line failure.

Required Knowledge

Topic 0 — a working shell (native terminal or WSL2).

Core Concepts

Navigation

- `pwd` — print the current directory
- `cd foldername` to move in, `cd ..` to move up, `cd ~` to jump home
- `ls` to list contents; `ls -la` for hidden files and details

Linux Filesystem Basics

- Everything hangs off one root: `/` — there are no separate drive letters like Windows' C:
- `~` is shorthand for your home directory; `.` is the current directory, `..` is the parent
- Absolute paths start with `/` and always resolve the same way; relative paths depend on where you currently are — this distinction is the single most common source of "file not found" confusion
- A few directories you'll see referenced later: /etc (system config), /usr (installed programs), /var (logs and variable data) — enough to be oriented, not a full OS course

File & Folder Operations

- `mkdir` to create a folder, `touch file.txt` to create an empty file
- `cp` to copy; `mv` to move or rename
- `rm` to delete a file; `rm -r` for a folder
- rm and rm -r have no undo, no trash bin, and no confirmation prompt by default. Always run pwd and ls before deleting, and double-check the exact path.

Permissions Basics

- `ls -l` shows a permission string like `-rwxr-xr--`: read, write, execute for owner / group / everyone else
- `chmod +x script.sh` makes a file executable — the fix the first time a downloaded script refuses to run
- You don't need deep permissions theory yet — just enough to recognize "Permission denied" and know chmod is the tool

Running Programs From the Terminal

- `javac File.java` to compile, `java File` to run a compiled Java class
- `node file.js` to run a JavaScript file directly with Node.js
- Project start scripts like `npm run dev` or `mvn spring-boot:run` start appearing once Part A and Part B begin — the command shape is worth recognizing now even though you won't run these yet

PATH & Environment Variables

- PATH is the list of folders the shell searches for a command's executable
- `echo $PATH` to view it; "command not found" almost always means a PATH problem, not a missing install
- `export VAR=value` sets a variable for one session; adding it to your shell profile (e.g. .zshrc) makes it permanent — this same pattern reappears later with .env files and Docker environment variables

Piping & Redirection

- `|` sends one command's output into the next command as input — e.g. `ls -la | grep .java`
- `>` writes output to a file, overwriting it; `>>` appends to a file instead
- `grep` searches text for a pattern; `wc -l` counts lines — together with pipes, this is how you'll read application logs starting in Part A

A Command-Line Debugging Method

- Read the exact error message first, top to bottom, before touching anything else
- Isolate the smallest command that reproduces the failure — don't debug inside a 10-step script
- Change one thing at a time and re-run — this is the same discipline you'll apply with IntelliJ's debugger in Topic 2

A Note on Windows

- If you set up WSL2 in Topic 0, everything above applies directly — you're already using a real Unix shell
- Plain CMD/PowerShell equivalents exist (dir for ls, del for rm, echo %PATH% for PATH) but are a fallback only, not a parallel track worth mastering

Hands-On Exercises

1. Chain pwd, cd, and ls -la to move two folders deep and confirm your location at each step
2. Create a folder with mkdir, a file inside it with touch, then list it with ls -la
3. Run chmod +x on a throwaway script and confirm it can now be executed
4. Pipe `ls -la | grep .java` against a folder containing mixed file types

Mini Projects

Folder Structure Builder — Steps: mkdir a project folder → cd into it → mkdir src, test, and docs subfolders → touch a README.md inside it → ls -la to confirm the structure.

Compile-and-Run Drill — Steps: write a one-file HelloTerminal.java → cd to its folder → javac HelloTerminal.java → java HelloTerminal → confirm the .class file appeared with ls.

PATH Investigation — Steps: run which java → print PATH and locate that folder in the list → temporarily remove it from PATH in one session → confirm java stops being found → restore PATH.

Log Grep & Redirect Drill — A realistic taste of reading application logs, which you'll do constantly starting in Part A. Steps: generate a text file with 30 mixed lines including some containing ERROR → grep ERROR file.log to isolate only the failing lines → count them with `grep ERROR file.log | wc -l` → redirect the filtered lines into a new file with `>`.

Common Mistakes

- Running rm -r without checking pwd first — deleting the wrong folder with no undo
- Confusing relative and absolute paths, especially right after a cd
- Treating a PATH problem as if the tool were never installed at all

Official Documentation

- MDN: Command Line Crash Course
- GNU Coreutils Manual — gnu.org/software/coreutils/manual/
- Microsoft: WSL Documentation — learn.microsoft.com/en-us/windows/wsl/

Self-Check Quiz

1. What's the difference between ~ and /?
2. What does chmod +x do, and when would you need it?
3. What's the difference between > and >>?
4. How would you filter a file's output down to only lines containing one keyword?
5. What's the first thing to check when the shell says "command not found"?

> **Reflection:** Describe one command that didn't do what you expected. What did that teach you about how the shell actually interprets what you type, rather than what you meant?

Completion Checklist

- [ ] Comfortable navigating and modifying the filesystem without a GUI
- [ ] Can explain absolute vs. relative paths and read a basic permission string
- [ ] Compiled and ran a Java program entirely from the terminal
- [ ] Used a pipe and redirection to filter and save command output
- [ ] Quiz self-checked and reflection written

> **Bridge to Part A:** Every Spring Boot run, every Docker command, and every deployment script in later parts is typed into this same terminal — the navigation and debugging habits built here don't change, only the commands do.

Topic 2 — Java Anatomy & Syntax

Estimated Duration: 1–1.5 weeks

Subtopics: keywords & access modifiers · naming conventions · variables & primitives · arrays · operators · control flow · exception handling · Scanner & console input · methods · compiling & packages · stack traces & the debugger

Before OOP, the raw syntax needs to be second nature — so a line like `public static void main(String[] args)` stops looking like a magic incantation and starts looking like five separate, understandable decisions.

Learning Objectives

Read any short Java method and explain what every keyword does; declare, iterate, and reason about arrays; write an if/else, a switch, and all three loop types from memory; wrap risky code in try/catch and explain checked vs. unchecked exceptions; read a stack trace and locate the failing line without help.

Required Knowledge

Topic 0 — JDK and IntelliJ installed. Topic 1 — comfortable compiling and running from the terminal.

Core Concepts

Anatomy of a Java Program

Token	What It Means
public	Access modifier — visible from anywhere, including other packages
class HelloWorld	Declares a class; the filename must match this name exactly
static	Belongs to the class itself, not an instance — the JVM calls main() with no object created
void	This method returns no value

main	The exact method name the JVM looks for as the program's entry point
(String[] args)	An array of command-line arguments; empty if none are given
System.out.println(…)	Prints a line of text to standard output

Keywords & Access Modifiers

- Access modifiers: public, private, protected, and default (no keyword, package-private)
- static vs. instance: static belongs to the class; instance members need an object created with new
- final locks a variable's value, a method against overriding, or a class against subclassing

Naming Conventions

- Classes & interfaces: PascalCase — Car, PaymentProcessor
- Methods & variables: camelCase — getBalance(), totalPrice
- Constants (static final): UPPER_SNAKE_CASE — MAX_USERS
- Packages: all lowercase, reverse-domain style — com.shopflow.service

Variables & Primitive Types

- Primitives: int, long, double, float, boolean, char, byte, short — stored by value
- Reference types: String, arrays, and any object — stored as a reference to memory
- var (Java 10+) lets the compiler infer the type from the right-hand side

Arrays

- Declaration: `int[] nums = new int[5];` or a literal: `int[] nums = {1, 2, 3};`
- Zero-indexed: nums[0] is the first element; nums.length gives the size
- Iterate with a standard indexed for-loop, or an enhanced for-each when the index isn't needed
- Arrays have a fixed size once created — this is exactly why collection classes like ArrayList exist in Phase 1

Operators

- Arithmetic: + - * / % ; integer division vs. floating-point division
- Relational: == != > < >= <= ; == compares references for objects, not content — use .equals()
- Logical: && || ! and short-circuit evaluation
- Assignment shorthand: += -= *= /= ++ –

Control Flow

- if / else if / else, and the ternary operator (condition ? a : b)
- switch statements and modern arrow-syntax switch expressions (Java 14+) — prefer arrow syntax; recognize the older colon syntax only because you'll see it in existing codebases
- Loops: for, enhanced for-each, while, do-while
- break / continue, and labeled loops for nested breaks

Exception Handling

- try / catch / finally isolates code that might fail and handles it without crashing the program
- Checked exceptions (must be declared or caught, e.g. IOException) vs. unchecked (RuntimeException and subclasses, e.g. NullPointerException)
- throw raises an exception yourself; throws in a method signature declares one it might produce
- A simple custom exception is just a class extending Exception or RuntimeException

Scanner & Console Input

- `Scanner sc = new Scanner(System.in);` reads from standard input
- sc.nextLine(), sc.nextInt(), sc.nextDouble() for typed input
- Validate with hasNextInt() before consuming, and catch InputMismatchException

Methods

- Declaration: modifier returnType methodName(paramType paramName) { … }
- Overloading: same name, different parameter lists
- Passing by value: primitives are copied; object references are copied but point to the same object

Compiling, Running & Packages

- JDK to write/compile, JRE to run, JVM as the engine that executes bytecode
- javac HelloWorld.java compiles to HelloWorld.class; java HelloWorld runs it
- package at the top of a file, import to use classes from other packages
- Comments: //, /* */, and /** */ Javadoc comments for public APIs

Reading Stack Traces & Using a Debugger

- Read a stack trace top-down: the exception type and message, the exact file/line it was thrown from, then the call chain that led there
- Common early exceptions to recognize on sight: NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, ClassCastException
- Set a breakpoint in IntelliJ and run in Debug mode instead of Run mode
- Step over vs. step into a method call, and inspect variable values in the debugger panel while paused

Hands-On Exercises

1. Declare one variable per primitive type and print each with a label
2. Write one if/else chain, one switch expression, and all three loop types in a single scratch file

3. Manually throw and catch one exception with a custom message

Mini Projects

Array & Scanner Quiz App — Steps: prompt the user with Scanner for 3 quiz questions → store answers in a String[] array → compare against a correct-answers array → print a score out of 3.

Overload Practice — Steps: write add(int,int) → write add(double,double) → write add(String,String) for concatenation → call all three from main() and print results.

Command-Line Calculator (with real error handling) — Steps: parse args[0] and args[2] with Integer.parseInt → read the operator from args[1] → switch on the operator → wrap the divide operation in try/catch for ArithmeticException → also catch NumberFormatException around the parsing.

Stack Trace Autopsy — Steps: deliberately write code that throws a NullPointerException, then an ArrayIndexOutOfBoundsException → read each stack trace top to bottom and identify the exact failing line before fixing it → repeat the same exercise using the IntelliJ debugger with a breakpoint instead of just reading output.

Common Mistakes

- Using == to compare Strings or objects instead of .equals()
- Off-by-one errors at array bounds (nums[nums.length] is always out of range)
- Catching Exception too broadly instead of the specific exception you expect
- Forgetting break in an old-style switch statement and falling through unintentionally

Official Documentation

- Oracle: The Java Tutorials
- Oracle: Java Language Basics
- dev.java (official Java developer site)
- Baeldung: Java Exceptions
- Java SE API Documentation

Self-Check Quiz

1. What’s the difference between a checked and an unchecked exception?
2. What typically causes a NullPointerException?
3. When would you reach for an enhanced for-each instead of an indexed for-loop?
4. What does static actually mean for a method or field?
5. What does the top line of a stack trace tell you, specifically?
6. Why does == fail to compare two equal Strings correctly?

> **Reflection:** Pick one stack trace you personally triggered this week and explain, in your own words, exactly what caused it and how you found the fix.

Completion Checklist

- [ ] Can read any short Java method and explain every keyword
- [ ] Declared and iterated over an array without help
- [ ] Wrote all three loop types and both switch styles from memory
- [ ] Wrapped risky code in try/catch and can explain checked vs. unchecked
- [ ] Read a stack trace and found the failing line unaided
- [ ] Quiz self-checked and reflection written

> **Bridge to Part A:** This raw syntax is exactly what sits inside every Spring Boot controller and service method in Part A — nothing new syntactically, just new annotations wrapped around code that looks like this.

Topic 3 — Git & GitHub Essentials

Estimated Duration: 1 week

Subtopics: commit lifecycle · branching & naming · merge vs. rebase · conflict resolution · .gitignore & secrets · undoing changes · commit conventions · pull requests & review · issues & boards · branching strategy · branch protection · authentication · command reference

Your very first Phase 1 mini-project gets committed to Git, so this comes before OOP, not after. Master the core loop now — init/clone, add, commit, push/pull, branch, merge, pull requests — and come back to rebase, cherry-pick, and team branching strategy once the basics feel automatic.

Learning Objectives

Perform the full add / commit / push / pull / branch / merge loop without hesitation; resolve a merge conflict by hand; open, review, and merge a pull request, including from the GitHub CLI; explain what NOT to do — force-pushing shared history, committing secrets — and why.

Required Knowledge

Topic 1 — comfortable in the terminal.

Core Concepts

Git Basics & the Commit Lifecycle

- Git is a distributed version control system — every commit is a full snapshot, and every developer holds the complete history locally
- The core loop: git init/clone, edit files, git add to stage, git commit to snapshot, git log/diff to inspect history
- If a laptop is lost mid-sprint, nothing is lost — cloning the repository restores the entire history instantly

Branching & Naming Conventions

- A branch is an isolated line of development — build and break things without touching what everyone else depends on
- Prefixes like feature/, bugfix/, hotfix/ let anyone glancing at the repo instantly understand a branch's purpose

Merging vs. Rebasing

- A merge keeps the full, real history of how branches diverged and came back together
- A rebase rewrites your branch's commits on top of the latest main for a clean, linear history — at the cost of the original divergence record
- Many teams configure GitHub's "squash and merge" as the only allowed strategy, so every feature lands on main as one clean commit regardless of how messy its history was

Resolving Merge Conflicts

- A conflict happens when two branches change the same lines of the same file and Git can't auto-resolve it — completely normal on any team larger than one
- Git marks the conflicting lines directly in the file; you edit them to the agreed final version and commit

.gitignore & Keeping Secrets Out of Git

- A .gitignore file tells Git which files to never track — build output, dependency folders, and critically, files holding passwords or API keys
- Once a secret is committed, it stays in the project's history forever unless that history is rewritten — treat prevention as the only real fix

Undoing Changes: Stash, Reset & Revert

- git stash temporarily shelves uncommitted changes
- git reset rewrites history — dangerous the moment anyone else has already pulled it
- git revert creates a new commit that undoes an earlier one, leaving history intact and auditable — this is what teams use to undo a bad deploy on production

Commit Message Conventions

- A structured format like `feat: add coupon validation` or `fix: correct tax rounding error` keeps history scannable and lets tooling auto-generate changelogs

Pull Requests & Code Review

- A pull request formally proposes merging one branch into another; at least one other engineer reviews it before it reaches main
- Almost every company requires at least one approving review before a PR can merge — this is often a new hire's first real lesson in a codebase's conventions

Issues, Boards & Branching Strategy

- Issues track bugs, features, or tasks with discussion attached directly to the code they concern; Project boards arrange issues Kanban-style
- Git Flow uses long-lived develop/release/hotfix branches for scheduled releases; GitHub Flow (trunk-based) keeps a single always-deployable main with short-lived feature branches — the norm at fast-moving SaaS teams

Branch Protection & Authentication

- Branch protection can block direct pushes to main for everyone, requiring a PR, passing checks, and approvals — even for the team's most senior engineer
- SSH keys authenticate a person without typing a password on every push; Personal Access Tokens (PATs) authenticate scripts and CI, and can be scoped and revoked individually

Getting Unstuck With Git

- `git <command> --help` (or `git help <command>`) opens the official manual page for exactly the command you're on
- `git reflog` is a safety net — it records every place HEAD has pointed, so a commit that looks "lost" after a bad reset is almost always still recoverable
- When an AI assistant suggests a git command, ask it to explain what the command does before running it — git is one of the few tools where a wrong guess can destroy work

Command Reference

Command	What It Does
git init / git clone	Create a new repository, or copy a remote one with full history
git status / git log –oneline	Show current state, or condensed commit history
git add . / git commit -m "msg"	Stage all changes, then save a snapshot with a message
git branch / git switch -c <name>	List branches, or create and switch to a new one
git merge / git rebase	Combine another branch in, or replay commits on top of it
git stash / git stash pop	Shelve uncommitted changes, then restore them later
git reset –hard	Discards uncommitted changes permanently — no undo
git revert <commit>	Safely undo a commit by creating a new, opposite commit
git push -u origin <branch> / git pull	Upload commits and set the default upstream / fetch and merge
git reflog	Show every place HEAD has pointed — the recovery net for a bad reset
gh pr create / gh pr merge	Create or merge a pull request from the terminal (GitHub CLI)

> `git reset --hard` permanently discards uncommitted changes. Always run git status first, and prefer git revert on any commit that's already been pushed or shared.

Hands-On Exercises

1. Run git init, then git add and git commit three separate times on a scratch folder
2. Run git log –oneline and read the condensed history back
3. Stash a change with git stash, confirm it's gone from git status, then restore it with git stash pop

Mini Projects

Repo From Day One — Steps: git init in your first mini-project folder → commit after every working increment → write a one-line README → push it to a new GitHub repo.

Solo Feature Branch + Conventional PR — Steps: create a feature/ branch for one concept project → commit your work using feat:/fix:/refactor: prefixes → open a PR against your own main → read your own diff like a reviewer would, then merge.

Merge Conflict Simulation — Steps: edit the same file/line on two branches → try merging one into the other → read the conflict markers → resolve manually and commit the resolution.

Branch-Protected Repo + CLI Workflow — Steps: create a new GitHub repo and turn on branch protection for main → require a PR and one passing check before merge → confirm a direct push to main is now blocked → install and authenticate the gh CLI, then create and merge one PR entirely from the terminal.

Common Mistakes

- Committing a .env file or API key before .gitignore is in place
- Force-pushing a branch other people have already pulled
- Writing vague commit messages like "fixed stuff" instead of a conventional prefix and a real description
- Starting new work on a stale local branch without pulling first

Official Documentation

- Pro Git Book (free, official) — git-scm.com/book/en/v2
- GitHub Docs — docs.github.com/en
- Learn Git Branching (interactive) — learngitbranching.js.org
- GitHub CLI Documentation — cli.github.com/manual

Self-Check Quiz

1. What's the practical difference between a merge and a rebase?
2. What does .gitignore actually prevent, and what doesn't it undo?
3. How does git revert differ from git reset, and why does that matter on a shared branch?
4. What is a pull request for, beyond just combining code?
5. What's the difference between an SSH key and a Personal Access Token?
6. What is git reflog for, and when would you reach for it?

> **Reflection:** Describe a moment — real or from the merge-conflict project — where Git either prevented you from losing work, or you had to recover from a mistake. What would you do differently next time?

Completion Checklist

- [ ] Completed the full add/commit/push/pull/branch/merge loop unaided
- [ ] Resolved a merge conflict by hand
- [ ] Opened, reviewed, and merged a pull request
- [ ] Created and merged a PR using the GitHub CLI
- [ ] Can explain why reset –hard and committed secrets are dangerous
- [ ] Quiz self-checked and reflection written

> **Bridge to Part A:** Every ShopFlow backend change from Part A onward goes through this exact branch → PR → review → merge loop, with branch protection on main starting now, not later.

Topic 4 — UML Basics

Estimated Duration: 3–4 days

Subtopics: use case diagrams · activity diagrams · sequence diagrams · class diagrams (association only) · ER diagrams

UML (Unified Modeling Language) is a standardized way to visualize how a system is structured and behaves, before or alongside writing the code. The order below follows how a real feature actually gets designed: what the system does, how a process flows, how objects talk to each other, what structure that implies, and finally what data it needs to persist.

Learning Objectives

Read and draw a use case diagram from a feature description; model a business process as an activity diagram; trace an API request across layers with a sequence diagram; draw a class diagram scoped to plain association; map real entities and relationships into an ER diagram.

Required Knowledge

Topic 2 — enough vocabulary to name classes, attributes, and methods.

Core Concepts

Use Case Diagrams — What the System Does

- Actors (stick figures) represent users or external systems
- Ovals represent actions the system supports (“Place Order”, “Cancel Subscription”)
- Good for scoping a feature with a non-technical stakeholder before any code exists — this is why it comes first

Activity Diagrams — How a Process Flows

- Flowchart-style diagram for business logic: start node, decision diamonds, end node
- Good for modeling a multi-step process like checkout or order fulfillment, once the use cases that trigger it are already known

Sequence Diagrams — How Objects Talk

- Vertical lifelines represent objects or services, time flowing top to bottom
- Horizontal arrows represent method calls or messages between them
- Extremely useful for mapping an API request across Controller → Service → Repository → DB, once the activity behind it is understood

Class Diagrams — The Structure Implied Above (Association Only, For Now)

- Boxes represent classes: name, attributes, and methods in three stacked sections
- For now, use plain association lines only. Aggregation (hollow diamond), composition (filled diamond), and inheritance (hollow triangle) will make far more sense once Phase 1 introduces OOP — revisit this diagram then and add them
- Multiplicity notation (1, 0..1, *, 1..*) describes how many of one class relate to another

ER Diagrams — The Data Behind It All

- Not strictly UML, but the same visual thinking applied to database schemas
- Entities become tables, attributes become columns, lines show foreign key relationships and cardinality
- The natural bridge into Part A’s Databases topic — draw this before writing a single CREATE TABLE

Hands-On Exercises

1. Draw one actor with two use cases by hand or in Mermaid
2. Draw a 3-step activity diagram for any everyday process (e.g. making coffee) with one decision diamond

Mini Projects

Use Case Diagram: Checkout Flow — Steps: draw a Customer actor → add ovals for Browse, Add to Cart, Apply Coupon, Checkout → connect each to the actor → add a Payment Provider as a second actor for the payment use case.

Activity Diagram: Order Fulfillment — Steps: add a start node → add decision diamonds for payment success/failure → add activity boxes for each state transition → add an end node for both success and failure paths.

Sequence Diagram: Login Request — Steps: draw lifelines for Client, Controller, Service, Repository, DB → add the login request arrow from Client → add each internal call in order → add the JWT response arrow back to Client.

Class Diagram: ShopFlow Product & Cart (association only) — Steps: list Product and Cart as two plain classes with attributes only, no inheritance yet → connect them with a simple association line and multiplicity (one Cart relates to many CartItems) → revisit this same diagram after Phase 1 and add inheritance/composition once you’ve learned them → then implement the code and compare.

ER Diagram: ShopFlow Schema — Steps: list entities: users, products, categories, orders, order_items → add key attributes to each → draw foreign key lines between related entities → mark one-to-many vs. many-to-many relationships.

Common Mistakes

- Drawing inheritance or composition on a class diagram before OOP has actually been learned
- Confusing a sequence diagram with a plain flowchart — a sequence diagram is about who talks to whom, in order
- Forgetting multiplicity on a class diagram, which leaves the relationship’s meaning ambiguous

Official Documentation

- UML Diagrams Reference — uml-diagrams.org
- OMG: Official UML Specification — omg.org/spec/UML
- PlantUML Documentation (text-based UML) — plantuml.com

Self-Check Quiz

1. What’s the difference between a use case diagram and an activity diagram?
2. What does a filled diamond mean on a class diagram, versus a hollow one?
3. What does a multiplicity of 1..* mean?
4. Why reach for a sequence diagram instead of just describing an API call in prose?
5. What does an ER diagram entity map to in a real relational database?

> **Reflection:** Pick one ShopFlow feature. Which diagram type would you reach for first to explain it to a teammate, and why that one over the others?

Completion Checklist

- [ ] Drew a use case diagram from a feature description
- [ ] Drew an activity diagram with at least one decision point
- [ ] Drew a sequence diagram tracing a request across layers
- [ ] Drew a class diagram scoped correctly to association only
- [ ] Drew an ER diagram with foreign keys and cardinality marked
- [ ] Quiz self-checked and reflection written

> **Bridge to Part A:** The ER diagram drawn here becomes the actual schema Part A's Databases topic implements with real CREATE TABLE statements — don't skip it.

Topic 5 — Dev Tools, AI-Assisted Learning & Transitioning to Part A

Estimated Duration: 2–3 days

Subtopics: dev tools roundup · documentation habits · an AI-assisted learning workflow · capstone reflection · Part A readiness

This closing topic ties the roadmap together: the tools that sit around your code, the habits that keep you learning instead of just copying, and an honest check on whether Part A is the right next step.

Learning Objectives

Identify the right tool for a task from the standard toolkit; apply a repeatable AI-assisted learning workflow that builds skill rather than dependency; maintain documentation habits — READMEs, code comments, a TIL log — going forward; self-assess readiness for Part A using the master checklist.

Required Knowledge

Topics 0–4.

Core Concepts

Dev Tools Roundup

Category	Tools
IDE / Editor	IntelliJ IDEA (Java/Spring Boot), VS Code (frontend, general-purpose)
Java Version Mgmt	SDKMAN (macOS/Linux/WSL2) for installing and switching JDK versions
API Testing	Postman, Insomnia — test endpoints manually before wiring up the frontend
Database Client	DBeaver or pgAdmin (PostgreSQL), MongoDB Compass (MongoDB)
Diagramming	Mermaid (text-based), draw.io / diagrams.net, Lucidchart
Project Tracking	Trello, GitHub Projects, Notion
Code Quality	SonarLint (in-IDE), ESLint + Prettier (JS/TS)
Design / Mockups	Figma — useful once Part B's frontend work starts needing a visual target
Terminal Enhancement	WSL2 (Windows), Oh My Zsh (Unix) — a real Unix shell plus a nicer prompt
Git From the Terminal	GitHub CLI (gh) — create/merge PRs, manage issues, without leaving the terminal
API Documentation	Swagger/OpenAPI (auto-generated from Spring Boot endpoints)

Documentation Habits

- Write a short README for every project: what it does, why it exists, how to run it
- Comment code for why a decision was made, not what the code obviously already says
- Keep a running personal "TIL" (Today I Learned) note per topic — this is where every Reflection prompt in this roadmap should end up
- Read the official docs for a class or method before reaching for a tutorial or blog post about it

An AI-Assisted Learning Workflow

- Try writing the code yourself first — even a rough attempt is worth more than a correct answer you didn't produce
- If stuck, ask an AI assistant to explain the concept or the error, not to produce the final answer
- Once you have working code, ask an AI assistant to review it and explain any suggested change before accepting it
- Never commit or merge code you can't explain line by line, regardless of who or what wrote it
- Read a stack trace yourself first, then use an AI assistant to confirm or deepen your read of it — not as the first move

Getting Unstuck, Revisited

- The time-box-and-rotate habit from the front matter applies here too: 30–45 minutes stuck, log what you tried, switch topics, come back later
- By this point you should have several reflection entries — reread them; the same category of mistake showing up twice is worth fixing at the habit level, not just the code level

Hands-On Exercises

1. Write a README for one earlier mini project: what it does, why it exists, how to run it

2. Pick one error you personally hit this week, ask an AI assistant to explain (not fix) it, and write the explanation in your own words

Mini Projects

Toolkit Setup Drill — Steps: install and configure Postman or Insomnia, DBeaver, and the GitHub CLI → hit one public API with Postman/Insomnia and inspect the response → connect DBeaver to a local database instance and browse its tables.

AI Pair-Review Drill — Uses your Command-Line Calculator from Topic 2 as the review target. Steps: ask an AI assistant to review the Command-Line Calculator project → evaluate each suggestion individually before applying it → document which suggestions you accepted, which you rejected, and why.

Capstone Reflection & Part A Readiness Check — Steps: write a half-page reflection on Part 0: what was hardest, what you'd redo → reread your Reflection entries from Topics 0–4 and note any repeated mistake → complete the Master Progress Tracker on the final page before starting Part A.

Common Mistakes

- Accepting an AI-suggested code change without understanding why it works
- Skipping a README because a project "is just practice" — the habit is the point, not the project
- Treating dev tools as optional add-ons instead of setting them up now, while the stakes are still low

Official Documentation

- Postman Documentation — learning.postman.com
- DBeaver Documentation — github.com/dbeaver/dbeaver/wiki
- GitHub CLI Documentation — cli.github.com/manual

Self-Check Quiz

1. What's the difference between using AI to explain something versus using it to solve something?
2. Why write a README for a two-day practice project?
3. What is SonarLint for, and when does it run?
4. What's the risk of accepting an AI code suggestion you don't fully understand?
5. Name one tool from the roundup you'll use again in Part A's Databases topic.

> **Reflection:** This topic's reflection is the Capstone Reflection & Part A Readiness Check mini project above — complete it there rather than repeating it here.

Completion Checklist

- [ ] Postman/Insomnia, DBeaver, and the GitHub CLI installed and tested
- [ ] Completed an AI pair-review and documented accepted vs. rejected suggestions
- [ ] Wrote a README for at least one Part 0 project
- [ ] Capstone reflection written
- [ ] Reviewed the Master Progress Tracker on the final page and confirmed every topic is checked off

> **Bridge to Part A:** Part A — Backend Engineering assumes everything on the Master Progress Tracker is checked. Spring Boot introduces dependency injection, REST controllers, and a real database — all built on top of the JDK, terminal, Git, UML, and tooling habits from Part 0.

Master Progress Tracker

A consolidated view of every topic's Completion Checklist. Each topic's own checklist is the detailed version — this page is the fast, at-a-glance one to confirm before starting Part A.

Topic 0 — Environment Setup

- [ ] java -version and javac -version print matching versions
- [ ] JAVA_HOME is set and confirmed
- [ ] One shell environment chosen and used going forward
- [ ] IntelliJ IDEA installed; a one-file program runs and debugs successfully
- [ ] Quiz self-checked and reflection written

Topic 1 — Linux & Terminal Fundamentals

- [ ] Comfortable navigating and modifying the filesystem without a GUI
- [ ] Can explain absolute vs. relative paths and read a basic permission string
- [ ] Compiled and ran a Java program entirely from the terminal
- [ ] Used a pipe and redirection to filter and save command output
- [ ] Quiz self-checked and reflection written

Topic 2 — Java Anatomy & Syntax

- [ ] Can read any short Java method and explain every keyword
- [ ] Declared and iterated over an array without help
- [ ] Wrote all three loop types and both switch styles from memory
- [ ] Wrapped risky code in try/catch and can explain checked vs. unchecked
- [ ] Read a stack trace and found the failing line unaided
- [ ] Quiz self-checked and reflection written

Topic 3 — Git & GitHub Essentials

- [ ] Completed the full add/commit/push/pull/branch/merge loop unaided
- [ ] Resolved a merge conflict by hand
- [ ] Opened, reviewed, and merged a pull request
- [ ] Created and merged a PR using the GitHub CLI
- [ ] Can explain why reset –hard and committed secrets are dangerous
- [ ] Quiz self-checked and reflection written

Topic 4 — UML Basics

- [ ] Drew a use case diagram from a feature description
- [ ] Drew an activity diagram with at least one decision point
- [ ] Drew a sequence diagram tracing a request across layers
- [ ] Drew a class diagram scoped correctly to association only
- [ ] Drew an ER diagram with foreign keys and cardinality marked
- [ ] Quiz self-checked and reflection written

Topic 5 — Dev Tools, AI-Assisted Learning & Transitioning to Part A

- [ ] Postman/Insomnia, DBeaver, and the GitHub CLI installed and tested
- [ ] Completed an AI pair-review and documented accepted vs. rejected suggestions
- [ ] Wrote a README for at least one Part 0 project
- [ ] Capstone reflection written
- [ ] Reviewed the Master Progress Tracker on the final page and confirmed every topic is checked off

Next Step: Once every box above is checked, move on to Part A — Backend Engineering, where Spring Boot, REST APIs, and a real database build directly on everything set up in Part 0.